HALFBUILT: Website Build Specification

> Code Variant: Build 01 | Problem Statement 05: The Project Graveyard | Team ragini.pawar24 Live build: 24 October 2026, 12:00 PM to 11:59 PM IST. A Surprise Challenge arrives at the 6 hour mark.

You are helping our team build a frontend web app called **HALFBUILT** during a 12 hour hackathon. Read this whole file before writing any code. Build in the phases listed in section 13, one phase at a time, and make sure the app runs after every phase.

---

1. What we are building

HALFBUILT is a web platform where builders keep the projects they started but never finished, and decide what to do with each one next.

- Every project becomes a **building** in the user's own animated **city**.
- Completed parts of the project are **solid, lit floors**.
- Unfinished parts are **dashed scaffolding floors** on top.
- Projects left alone slowly **go dark** (windows switch off over time).
- A guided **Site Inspection** (6 questions) gives a **Revival Score from 0 to 100**, a **recommended path**, the reasoning, and **one small first step**.
- When a project is revived, a **crane** appears and the **lights switch back on**.

Around this core:

- A GitHub and LeetCode style **activity grid** on each profile.
- A **Revival Leaderboard** that ranks people by projects revived, not started.
- **Builders' Square**, a networking page to find co-builders for stalled projects.

Hard rule

This is **not a project management tool**. Do not add task boards, Kanban columns, sprints, due dates or deadlines. Every feature must support reflection on a stopped project or reviving it.

---

2. Tech stack

Layer	Choice
Framework	React 18 + Vite (JavaScript, not TypeScript, to stay fast)
Styling	Tailwind CSS v4 via `@tailwindcss/vite`
Animation	Framer Motion (`motion` package or `framer-motion`)
Routing	React Router v6
State	Zustand with the `persist` middleware (saves to localStorage)
Buildings	Hand written SVG generated from project data
Icons	lucide-react (optional)
Deploy	Vercel

No backend. No external APIs. Everything runs in the browser.

---

3. Folder structure

```
src/
  main.jsx
  App.jsx                routes + layout
  index.css              Tailwind import + CSS variables (design tokens)
  store/
    useStore.js          Zustand store, all actions live here
    seed.js              demo users, projects and activity events
  logic/
    revivalScore.js      score + path + reasoning + first step
    activity.js          activity grid counts
    leaderboard.js       ranking
    buildingState.js     window lighting + status visuals
  components/
    Navbar.jsx           name top left, menu button top right
    MenuOverlay.jsx      full screen animated menu
    HelloIntro.jsx       multilingual hello animation
    Building.jsx         one SVG building
    City.jsx             row of buildings (skyline)
    ActivityGrid.jsx
    Timeline.jsx
    ScoreRing.jsx
    ProjectCard.jsx      used by list view and Builders' Square
    ProtectedRoute.jsx
  pages/
    Landing.jsx
    Login.jsx
    Signup.jsx
    MyCity.jsx
    AddProject.jsx
    ProjectDetail.jsx
    Inspection.jsx
    Leaderboard.jsx
    BuildersSquare.jsx
    Profile.jsx          public profile of any builder
    NotFound.jsx
```

---

4. Routes

Route	Page	Login needed
`/`	Hello intro, then Landing	No
`/login`, `/signup`	Auth forms	No
`/city`	My City (current user)	Yes
`/project/new`	Add a project	Yes
`/project/:id`	Project detail + timeline	Yes
`/project/:id/inspect`	Site Inspection	Yes (owner only)
`/leaderboard`	Revival Leaderboard	No
`/square`	Builders' Square	No (joining needs login)
`/builder/:username`	Public profile and city	No
`*`	404 page styled as an empty construction plot	No

---

5. Design direction

We want the site to feel **artistic, animated and alive**, like an illustrated city, not a dashboard. Clarity always comes first.

> **Team note:** we have our own design inspiration. We will attach reference images. When images are attached, match their colours, fonts and mood and replace the default tokens below.

Default design tokens (put in `index.css` as CSS variables)

```css
```

```
:root {
  --paper: #F4F1EA;      /* page background */
  --ink: #1B1B1F;        /* text, solid floors */
  --dusk: #2B2D42;       /* night sky, dark panels */
  --mist: #CDCEC8;       /* secondary surfaces */
  --light: #F5B942;      /* warm window light, main accent */
  --muted: #8C8C88;      /* secondary text */
  --scaffold: #9A9A94;   /* dashed scaffolding lines */
}
```

- **Fonts:** a display serif for headings (for example "Instrument Serif" or "Fraunces" from Google Fonts) and a clean sans for body text (for example "Inter").
- **Layout:** site name **HALFBUILT** on the top left, menu button on the top right, generous white space, centred content on a clear grid.
- **Tone of copy:** warm and encouraging. Abandoned projects are "paused construction", never "failures" or "dead".

Motion rules

- Every animation should explain something (floors rise when parts are added, lights switch on when revived).
- Respect `prefers-reduced-motion`: when it is set, skip the intro and use simple fades.
- Keep animations short: 200 to 600 ms for UI, up to 1.5 s for the skyline entrance.

---

6. Screens in detail

6.1 Hello intro (`HelloIntro.jsx`)

- Full screen, centred word that cycles: **Hello, Namaste, Hola, Bonjour, Konnichiwa, Hallo** (about 350 ms each, crossfade with slight upward motion).
- Total length about 2.5 s, then the intro slides up to reveal the Landing page.
- A small "Skip" button bottom right.
- Show the intro only once per browser session (use `sessionStorage`, wrapped in try/catch).

6.2 Landing (`Landing.jsx`)

- Navbar: HALFBUILT top left, menu button top right.
- Hero headline: **"Every unfinished project deserves another floor."**
- Sub line: "Turn your abandoned projects into a living city, and decide which ones to bring back."

- Primary button: **Build my city** (goes to `/signup`, or `/city` if logged in).
- Below the hero: an animated skyline that rises floor by floor, with a few buildings showing scaffolding and one with a crane.
- Three short sections: **How it works** (Add, Inspect, Revive), a preview of the activity grid, and a preview of the leaderboard.

6.3 Menu overlay (`MenuOverlay.jsx`)

- Opens full screen with a staggered animation of large links: My City, Add Project, Leaderboard, Builders' Square, Profile, Log out (or Log in / Sign up).
- Close with an X button or the Escape key. Trap focus while open.

6.4 Sign up and Log in

- Sign up fields: name, username (unique), email, password, skills (comma separated chips).
- Log in: email + password.
- Client side only. Hash the password with SHA-256 using the Web Crypto API before storing. Show friendly errors (email taken, wrong password).
- This is demo authentication only; add a code comment saying so.

6.5 My City (`MyCity.jsx`)

- Header: avatar circle (initials), name, stats: buildings, revived, landmarks, floors added this month.
- **Activity grid** under the header (section 9).
- Toggle: **City view** / **List view**.
- **City view:** horizontal skyline of all the user's buildings on a ground line. Hover lifts a building slightly and shows a tooltip (name, status, score if inspected). Click opens the project.
- **List view:** cards with name, status, completion percent, reason, score. Search box plus filters by status and by reason.
- **Revisit prompt:** a small card that suggests one project to look at, chosen as the paused project with the oldest `lastTouchedOn` that has never been inspected.
- Empty state: an empty plot of land with text "Your city is waiting for its first building" and an **Add a project** button.

6.6 Add a project (`AddProject.jsx`)

Two column layout: form on the left, live building preview on the right that grows as the form is filled.

Fields:

- Name (required)
- What it is (short description)
- What you wanted it to become (goal)
- Completed parts (chips, each becomes a solid floor)
- Unfinished parts (chips, each becomes a scaffolding floor)
- Why it stopped (select): Ran out of time or priorities changed, The event ended, Scope grew too large, Got stuck technically, Lost motivation, The idea felt weak
- Started on (date), Last worked on (date)
- Tools or tech used (chips), Links (repo, demo)

On save: create the project with status `paused`, add a `created` activity event, and navigate to the project page with a short "building rises" animation.

6.7 Project detail (`ProjectDetail.jsx`)

- Left: large Building with a sign at its base showing the reason (for example "PAUSED: TIME").
- Right: name, description, goal, completed vs unfinished parts, tools, links, status badge, latest Revival Score.
- Actions depend on status:
  - `paused`: **Start Site Inspection**
  - `inspected`: show plan, buttons for the recommended path plus "Choose another path"
  - `reviving`: **Mark a part as done** (moves a part from unfinished to completed, adds a floor with animation and an activity event), and **Mark as rebuilt** when no unfinished parts are left
  - `landmark`: show the lessons learned plaque
- Toggle **Open for co-builders** with skills needed (lists it in Builders' Square).
- **Timeline** (construction log): vertical list of dated events, newest first.

6.8 Site Inspection (`Inspection.jsx`)

- One question per screen, progress bar styled as a scanner line moving up the building.
- Back and Next buttons, keyboard friendly.
- The 6 questions and answer options are defined in section 8.
- Result screen: animated **ScoreRing** counting up to the score, the recommended path in large text, the reasoning as 2 or 3 short sentences, the first step, and buttons:
  - **Accept plan** (applies the path, see section 8.4)
  - **Choose another path** (shows all five paths as cards)

6.9 Revival Leaderboard (`Leaderboard.jsx`)

- Podium for the top 3 (animated rise), then a ranked table.
- Columns: rank, builder, projects revived, floors added after revival, landmarks, points.
- Tabs: **All time** and **This month**.
- Highlight the current user's row.

6.10 Builders' Square (`BuildersSquare.jsx`)

- Grid of projects that are `openForCollab`, from all users.
- Each card: mini building, project name, owner, reason it stopped, skills needed (chips), **Request to join** button.
- Filter by skill and search.
- Request to join: small modal with a message. Store the request on the project (`joinRequests`). The owner sees requests on the project page with Accept / Decline. Accepted users are listed as co-builders.

6.11 Public profile (`/builder/:username`)

Same as My City but read only: header, activity grid, skyline.

---

7. Data model

```js
```

```
// User
{
  id, name, username, email, passwordHash,
  skills: [],            // ["React", "UI design"]
  joinedOn               // ISO date
}

// Project
{
  id, ownerId,
  name, description, goal,
  completedParts: [],    // strings, each = one solid floor
  pendingParts: [],      // strings, each = one scaffolding floor
  reason,                // "time" | "event" | "scope" | "technical" | "motivation" | "wea
  tools: [], links: { repo: "", demo: "" },
  startedOn, lastTouchedOn,
  status,                // "paused" | "inspected" | "reviving" | "rebuilt" | "landmark"
  inspection: null | { answers, score, path, reasons: [], firstStep, date },
  chosenPath: null | "continue" | "restart" | "changeIdea" | "repurpose" | "archive",
  lessons: "",           // for landmarks
  openForCollab: false, skillsNeeded: [],
  joinRequests: [],      // { userId, message, status: "pending" | "accepted" | "declined
  coBuilders: []         // userIds
}

// ActivityEvent
{
  id, userId, projectId,
  type,                  // "created" | "inspected" | "revived" | "floorAdded" | "rebuilt
  date,                  // ISO date
  detail: ""
}
```

Store shape: `{ users, projects, events, currentUserId }`. All changes go through store actions (`signup`, `login`, `logout`, `addProject`, `updateProject`, `runInspection`, `applyPath`, `completePart`, `markRebuilt`, `archiveProject`, `toggleCollab`, `requestJoin`, `respondJoin`). Every action that changes a project also updates `lastTouchedOn` and logs an ActivityEvent.

Components never keep their own copy of data. They read from the store.

---

8. Revival Score logic (`logic/revivalScore.js`)

Write this as pure functions with no React, so it can be tested and changed quickly.

8.1 Questions and points (total 100)

#	Question	Answers and points
Q1	How much of the project is done?	Slider 0 to 100 percent. Points = round(percent / 100 x 25). Default the slider to completedParts / (completed + pending).
Q2	Why did it stop?	Pre-filled from `reason`, editable. event 20, time 18, scope 14, technical 12, motivation 8, weakIdea 4
Q3	Do you still care about this idea?	1 to 5 scale. 1 = 0, 2 = 10, 3 = 18, 4 = 26, 5 = 30
Q4	Is the problem it solves still relevant?	Yes 15, Somewhat 8, No 0
Q5	How much effort is left?	Under a week 10, A few weeks 5, Months 2
Q6	Are any parts reusable elsewhere?	Yes / No (0 points, used for the path)

Weights: interest 30, completion 25, reason 20, relevance 15, effort 10.

Check: interest 4 (26) + 60 percent done (15) + time (18) + somewhat relevant (8) + a few weeks (5) = **72**. Use this as the demo example.

8.2 Path rules (check in this order, first match wins)

1. **changeIdea**: interest >= 3 AND (reason is weakIdea OR relevance is No)
2. **continue**: interest >= 4 AND completion >= 50
3. **restart**: interest >= 3 AND (completion < 50 OR last touched more than 12 months ago)
4. **repurpose**: reusable is Yes
5. **archive**: everything else

8.3 Reasoning and first step

Return 2 or 3 short reasons built from the answers, for example:

- "You finished 60 percent of it."
- "It stopped because of time, not because the idea was bad."
- "You still care about it."

First step templates (use the first item of `pendingParts` when present):

- continue: "Spend about 2 hours finishing: {first pending part}."
- restart: "Spend about 2 hours writing a smaller version 1 plan with only 3 parts."
- changeIdea: "Spend about 2 hours writing one new problem this project could solve."
- repurpose: "Spend about 2 hours moving {a completed part} into a new or existing project."
- archive: "Write three lessons you learned, then lay it to rest as a landmark."

8.4 Applying a path

- continue or restart: status becomes `reviving`, crane appears, lights on, event `revived`.
- changeIdea: open an edit form to rewrite the goal, then status `reviving`.
- repurpose: choose a completed part and a target project; copy it into the target's completed parts; source becomes `landmark`.
- archive: ask for lessons learned; status `landmark`; event `archived`.

---

9. Activity grid (`logic/activity.js` + `ActivityGrid.jsx`)

- Last 26 weeks, 7 rows (days) by 26 columns (weeks), like GitHub.
- Count the user's ActivityEvents per day.
- 5 levels: 0, 1, 2 to 3, 4 to 5, 6 or more. Colour levels from `--mist` to `--ink` (or the accent if our inspo says so).
- Tooltip on hover: "3 actions on 12 Oct 2026".
- Show current streak and total actions beside the grid.

10. Leaderboard logic (`logic/leaderboard.js`)

Points = revived projects x 10 + floors added after revival x 2 + landmarks x 3. Sort by points, then by revived count. "This month" counts only events in the current month.

11. Building component (`Building.jsx`)

Props: `project`, `floorHeight` (default 18), `width` (based on total parts, 48 to 90 px), `interactive`, `showSign`.

Drawing rules:

- One solid floor per completed part, stacked from the ground up. Each floor has a row of windows.
- One dashed scaffolding floor per pending part on top, with a light diagonal brace.
- **Window lighting** from days since `lastTouchedOn`:

    - under 30 days: most windows lit with `--light`
    - 30 to 180 days: about half lit
    - over 180 days: all dark
    - status `reviving` or `rebuilt`: all lit regardless of date
- `reviving`: crane on the roof (simple lines), gentle swing animation.
- `rebuilt`: no scaffolding, a small flag on the roof.
- `landmark`: building in a lighter stone tone with a plaque at the base.
- Sign at the base with the reason in small caps when `showSign` is true.

Animations:

- On mount, floors rise one by one (stagger 60 ms).
- When a part is completed, its scaffolding floor fills in solid and the windows light up.
- Hover lift of 4 px with a soft shadow when `interactive`.

Accessibility: `role="img"` and an `aria-label` like "Habit Tracker: 6 floors built, 3 unfinished, paused because of time".

---

12. Seed data (`store/seed.js`)

Load seed data only when the store is empty, so the demo never looks empty.

- 6 demo builders with skills.
- 3 to 6 projects each, mixed statuses (paused, reviving, rebuilt, landmark), realistic names (Habit Tracker, Campus Lost and Found, Recipe Remix, Pixel Portfolio, Study Buddy Bot, Budget Bee, and so on).
- About 200 activity events spread over the last 6 months so grids and leaderboards look alive.
- At least 5 projects open for co-builders.
- Include the demo account **demo@halfbuilt.app** / **demo1234** that owns the Habit Tracker example (60 percent done, reason time) for judges.
- Add a "Reset demo data" button in the profile menu.

---

13. Build phases (follow in order)

Time	Phase	Done when
12:00 to 13:00	1. Setup	Vite + Tailwind + Router + Zustand running, design tokens, Navbar, MenuOverlay, seed data loaded
13:00 to 15:00	2. Buildings and city	`Building.jsx` and `City.jsx` working from data, My City page with both views
15:00 to 16:00	3. Add project and detail	Form with live preview, project page, timeline
16:00 to 17:00	4. Site Inspection	Questions, score, path, reasoning, first step, apply path
17:00 to 18:00	5. Auth, landing, deploy	Sign up, log in, protected routes, landing page, **first deploy to Vercel**
18:00 to 21:00	6. Surprise Challenge + community	Handle the challenge first, then activity grid, leaderboard, Builders' Square
21:00 to 23:00	7. Polish	Hello intro, animations, responsive layout, accessibility pass
23:00 to 23:59	8. Ship	Test on desktop and mobile, final deploy, submit

Rule: static first, motion later. If time is short, cut from phase 7 before touching phases 1 to 5.

---

14. Surprise Challenge readiness

At 18:00 we will receive a new requirement. The code must make it easy to change:

- Questions, answer options, weights and paths live in one config object in `revivalScore.js`.
- Statuses and their visuals live in one map in `buildingState.js`.
- All data changes go through the store, so a new field means one model change plus one visual layer.
- When we paste the challenge, first propose a short plan that touches as few files as possible, then build it.

---

15. Responsive and accessibility checklist

- Works from 360 px phones to large desktops. On mobile the skyline scrolls sideways and forms are one column.
- Every page keyboard navigable with visible focus rings.
- List view always available as a non visual alternative to the city.
- Colour contrast at least WCAG AA for text.
- `prefers-reduced-motion` respected everywhere.
- All localStorage and sessionStorage reads and writes wrapped in try/catch.

---

16. Deploy

- `npm run build` must pass with no errors.
- Add `vercel.json` with a rewrite so React Router routes work on refresh:

```json
{ "rewrites": [{ "source": "/(.*)", "destination": "/" }] }
```

- Deploy to Vercel and share the public link.

---

17. Final acceptance checklist

- [ ] Intro plays once, skippable, then landing with name top left and menu top right
- [ ] Sign up, log in, log out, protected routes
- [ ] Add project; building grows live in the preview
- [ ] My City in city view and list view, search and filters
- [ ] Project detail with timeline and status actions
- [ ] Site Inspection gives score, path, reasons, first step; demo project scores 72
- [ ] Reviving shows a crane and lights; completing a part adds a floor
- [ ] Archive creates a landmark with lessons
- [ ] Activity grid, leaderboard, Builders' Square with join requests
- [ ] Demo account works, reset demo data works
- [ ] No task boards, deadlines or Kanban anywhere
- [ ] Mobile layout checked, reduced motion checked
- [ ] Deployed on Vercel, routes work on refresh

---

18. How to work with us

- Build one phase at a time and tell us what to run and test after each phase.

- Keep components small and readable; we must understand and explain every part to the judges.
- Do not add libraries beyond section 2 without asking.
- If something in this spec conflicts with our attached design images, follow the images for visuals and this spec for behaviour.